

DATA 260 Homework 3

Authentication, Session Management, and Retrieval-Only RAG

Student: Vrishin Dharmesh Kunnatham Parambath

Repository: <https://github.com/Viki2183/data260-8844>

Course: DATA 260

Submission tag: hw3

Final tagged commit: To be updated after the final report commit

Project summary. In this homework, I extended my previous vulnerability-report application with authentication and protected sessions. I also built a retrieval-only experiment that compares token, semantic, and sentence-window chunking techniques over a local corpus of open-source package vulnerability records.

Personal configuration

Value	Configuration
SID4	8844
PORT_BASE	8744
PREFIX	s8844
SEED	8844
VERIFY_SEED	268844
DOMAIN_ID	4
Assigned domain	Open-source package vulnerabilities
Python	3.14.0
Device	CPU
Embedding model	sentence-transformers/all-MiniLM-L6-v2

1. Project objective

The first part of the homework required me to add a simple authentication system to the FastAPI application I had already built in HW1 and HW2. The application is designed around open-source package vulnerability reports. The goal was not to replace the earlier application, but to extend it with login, logout, session management, a protected dashboard, and Bootstrap styling.

The second part focused on retrieval-only RAG. I used public OSV PyPI vulnerability records as the corpus and compared three different ways of splitting documents before embedding and retrieval. The comparison measured both retrieval quality and practical characteristics such as chunk count, chunk length, and latency.

2. Part 1: FastAPI authentication application

2.1 Application structure

I kept the original FastAPI vulnerability-report API in `code/web_application/main.py`. I added a separate router in `code/web_application/routers/auth.py` and moved the HTML pages into the `templates` directory. This structure keeps the authentication routes separate from the vulnerability-report API routes.

Route	Purpose	Expected behavior
/	Domain application home page	Shows the vulnerability-report interface and session-aware navigation
/login	Login page and credential validation	Valid credentials redirect to the dashboard; invalid credentials show an alert
/dashboard	Protected page	Only an active logged-in session can access it
/logout	Session destruction	Clears the session and redirects to the home page

2.2 Authentication logic

For this homework demonstration, I used the credentials `admin` and `password`. In a production system, credentials would be stored securely in a database and passwords would be hashed. The important part for this assignment was demonstrating the route behavior and session lifecycle.

```
if username == VALID_USERNAME and password == VALID_PASSWORD:
    request.session["user"] = username
    request.session["last_activity"] = time.time()

    return HttpResponseRedirect(
        url="/dashboard",
        status_code=HTTP_302_FOUND,
    )
```

Invalid credentials do not create a session. Instead, the login template is rendered again with an error message. The message appears as a Bootstrap danger alert.

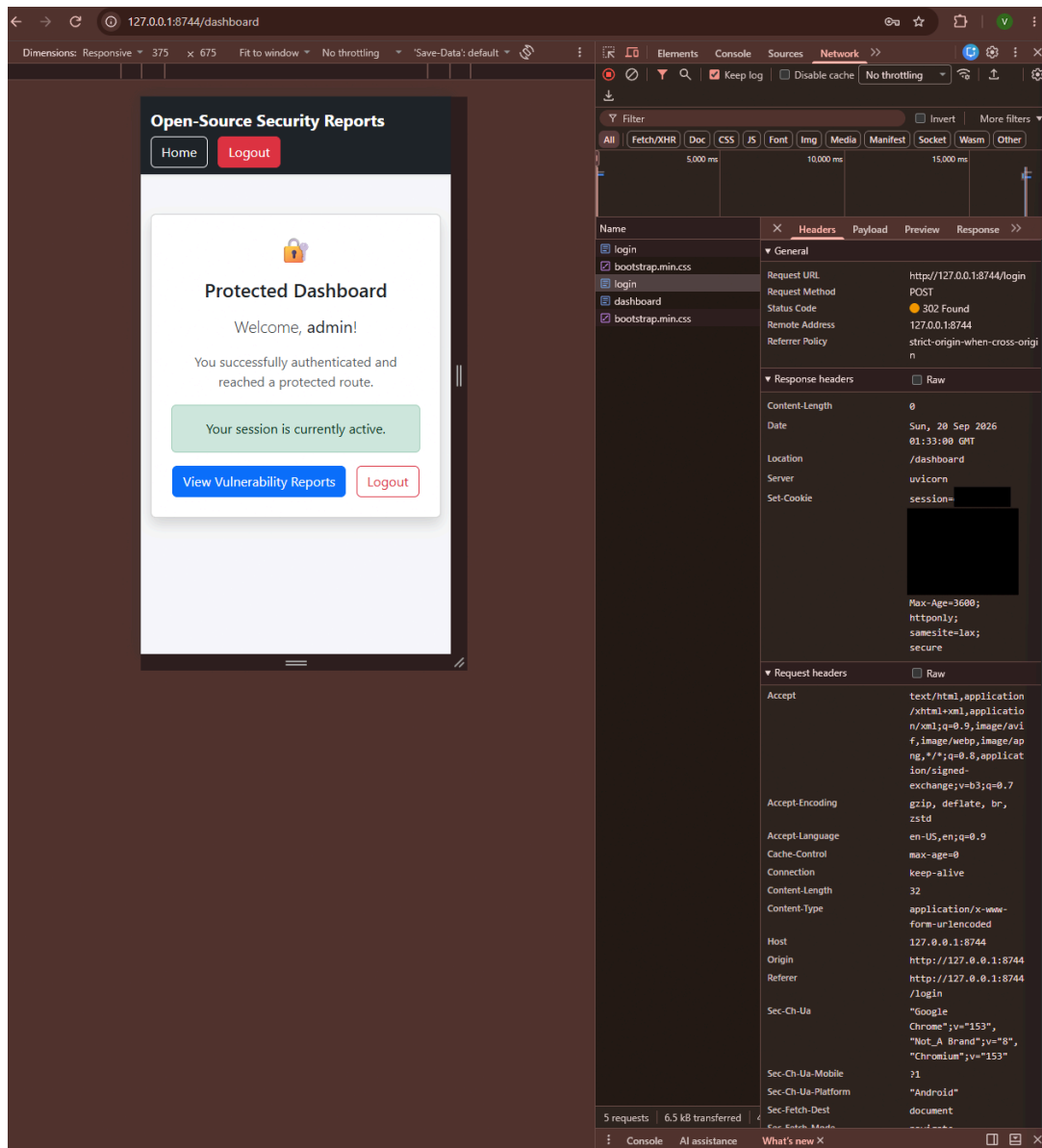
```
return templates.TemplateResponse(
    request=request,
    name="login.html",
    context={
        "user": None,
        "error": "Invalid username or password.",
    },
    status_code=401,
)
```

2.3 Session middleware and secure cookie

Starlette's SessionMiddleware signs the browser session cookie. I configured a secret key, a one-hour cookie lifetime, SameSite protection, and the Secure flag. The Secure flag is important because it prevents the browser from sending the cookie over an ordinary insecure connection in normal deployments.

```
app.add_middleware(
    SessionMiddleware,
    secret_key=SECRET_KEY,
    https_only=True,
    same_site="lax",
    max_age=3600,
)
```

I captured the POST login response in Chrome DevTools. The response returned status 302 and redirected to the dashboard. The Set-Cookie header contained Max-Age, HttpOnly, SameSite=Lax, and Secure attributes. I redacted the session value in the screenshot because the value itself should not be shared.



Secure session cookie and redirect after a successful login.

2.4 Idle session timeout

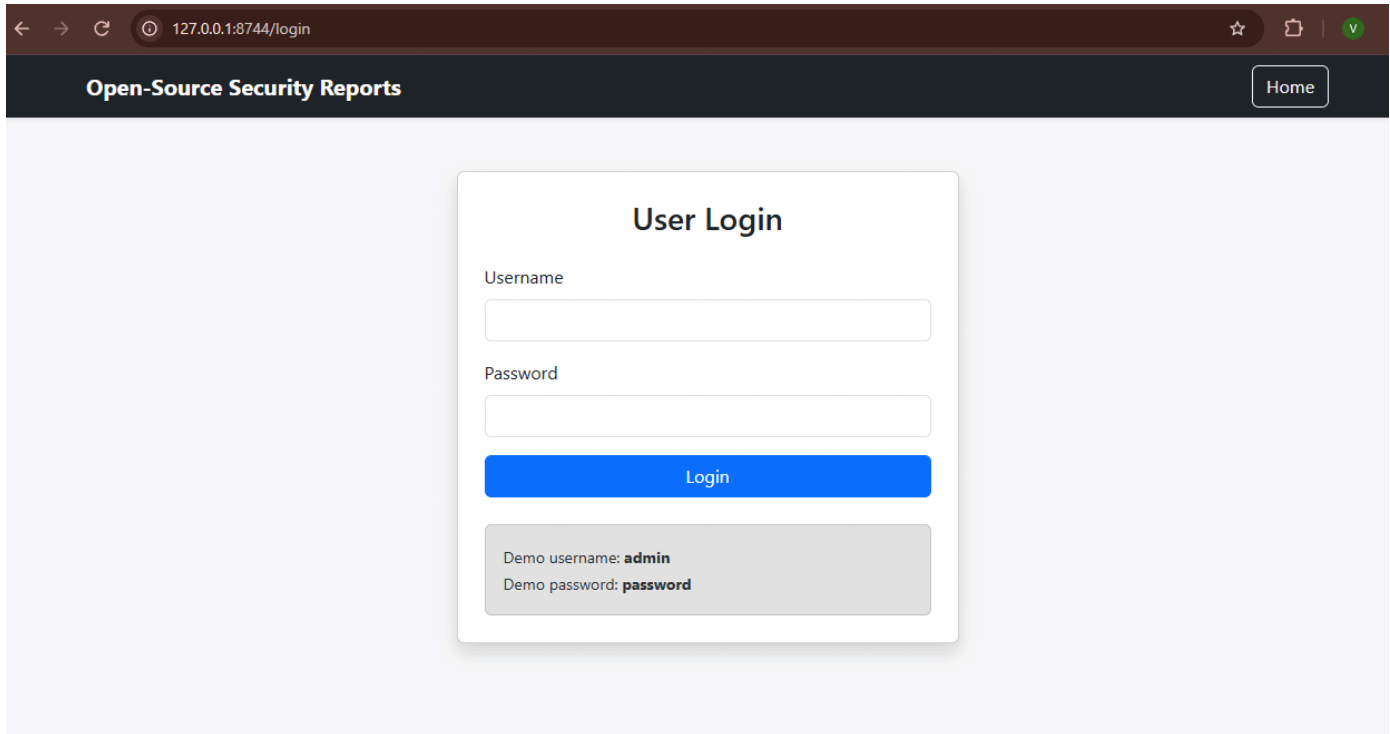
In addition to the cookie lifetime, I implemented an application-level idle timeout. Each session stores the time of the user's latest activity. When a protected route is requested, the application compares the current time with the stored activity time. If the idle period is longer than 900 seconds, the session is cleared.

```
if elapsed_time > IDLE_TIMEOUT_SECONDS:
    request.session.clear()
    return None

request.session["last_activity"] = time.time()
```

To test the behavior without waiting fifteen minutes, I temporarily changed the timeout to ten seconds. After logging in, I waited longer than ten seconds and then opened the dashboard directly. The application redirected

me to the login page, showing that an expired session could not be reused.

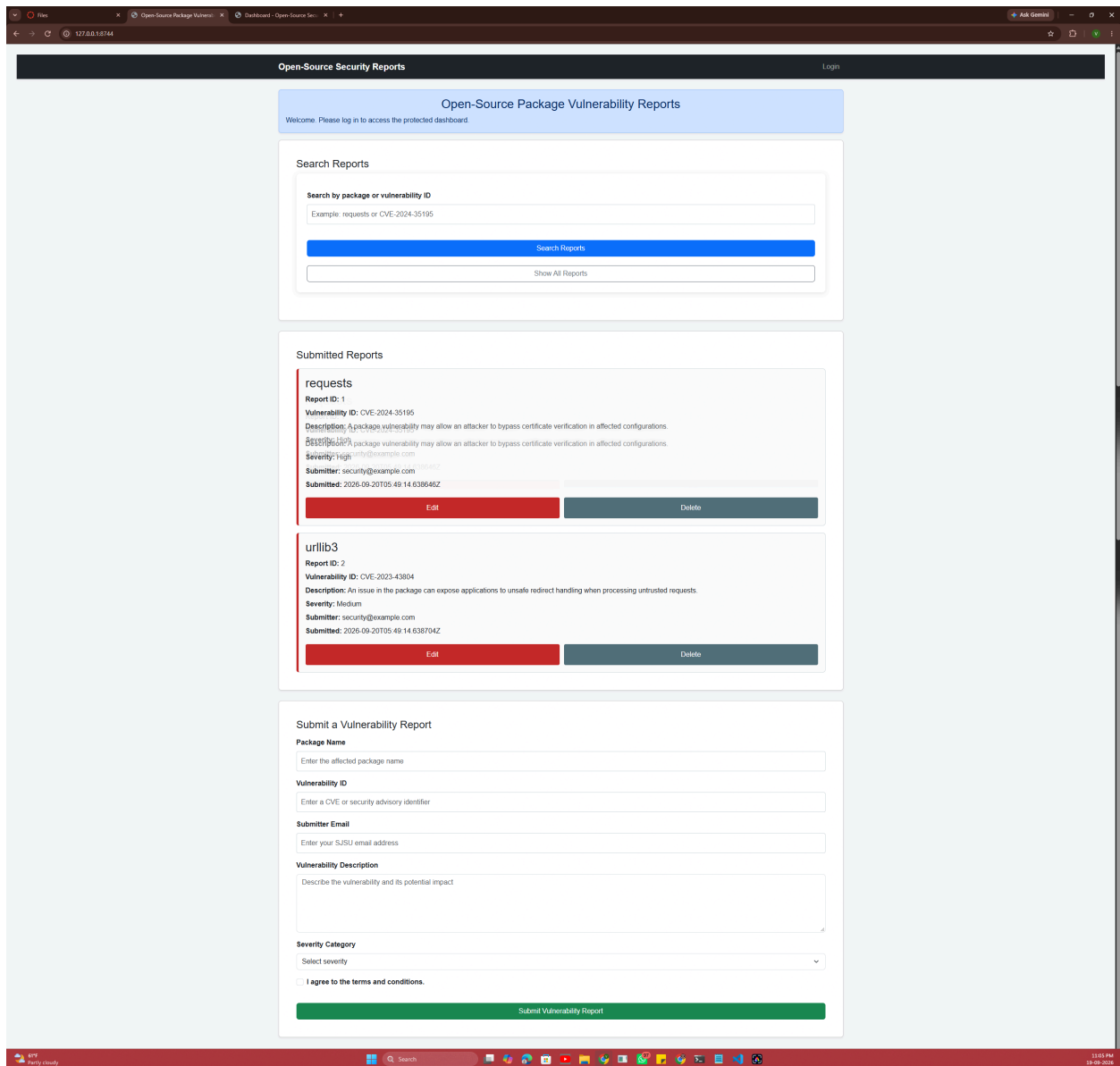


The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8744/login'. The page title is 'Open-Source Security Reports'. The main content is a white card titled 'User Login'. It contains two input fields labeled 'Username' and 'Password', a blue 'Login' button, and a grey box with the text 'Demo username: admin' and 'Demo password: password'.

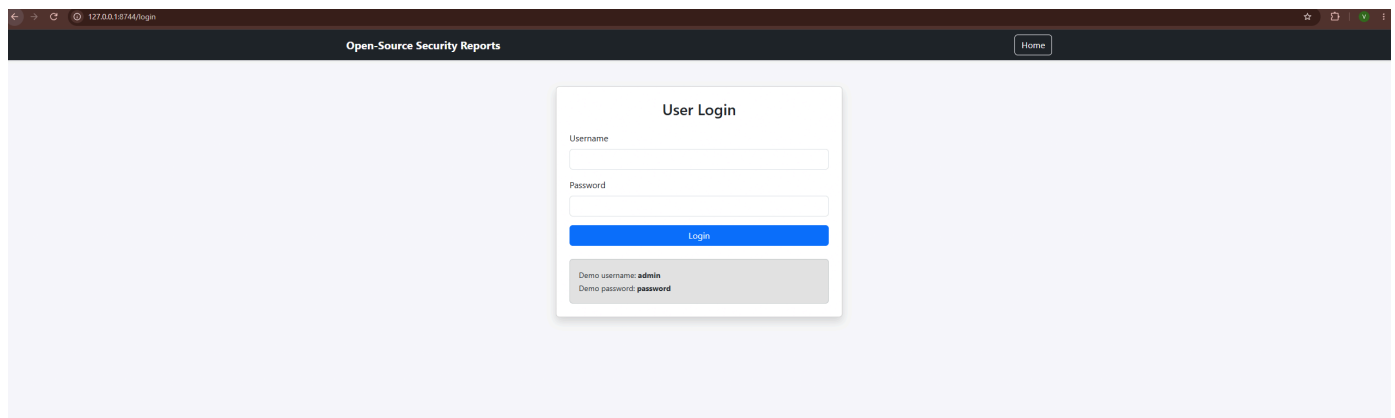
Expired session redirected to the login page.

2.5 Bootstrap styling

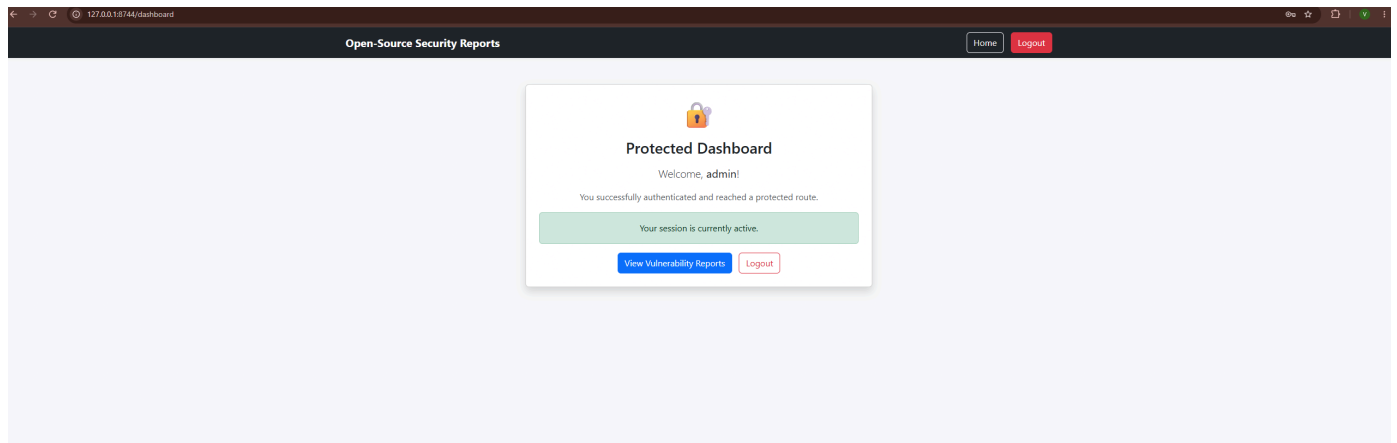
I used Bootstrap navigation bars, cards, alerts, buttons, form controls, and responsive containers. The home page preserves the HW2 vulnerability report functionality while adding a session-aware navigation bar. The login page uses a centered card, and the dashboard uses a protected-route card that displays the authenticated username.



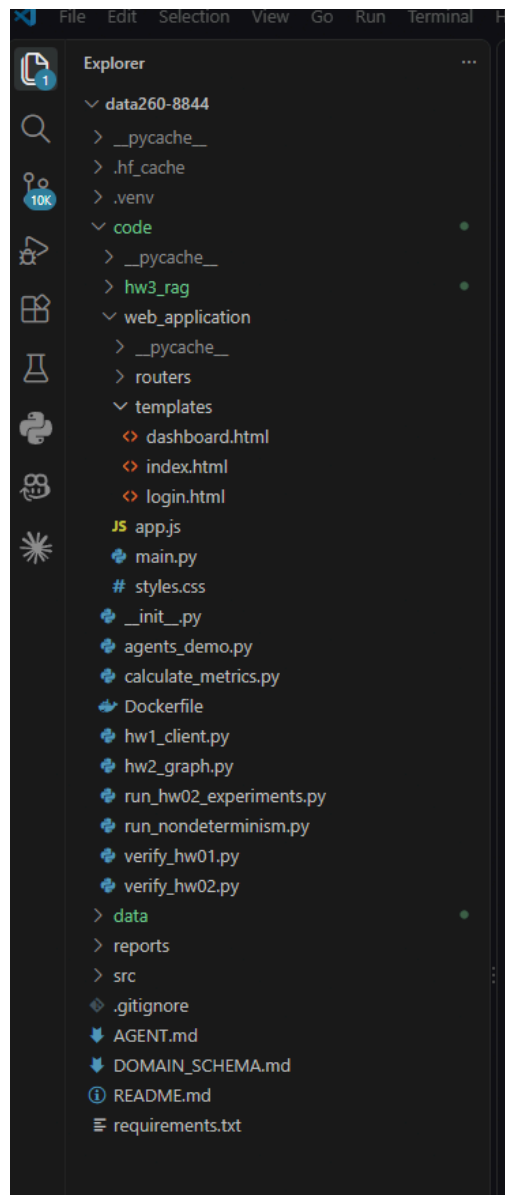
Home page showing the vulnerability-report application.



Bootstrap login page.



Protected dashboard after successful authentication.



Templates directory containing index.html, login.html, and dashboard.html.

3. Part 2: Retrieval-only RAG experiment

3.1 Corpus construction

My assigned domain was open-source package vulnerabilities. I downloaded the official OSV PyPI vulnerability dump and extracted the JSON records locally. The full source contained 25,641 records, so I created a deterministic subset rather than embedding the entire dump. The selection required a non-empty summary, at least 800 characters of details, and affected package information.

The final local corpus contains 69 JSON records and 507,649 bytes. Each record retains advisory identifiers, aliases, summaries, detailed descriptions, affected packages, version ranges, references, severity information, and other OSV metadata. The source URL, access date, file sizes, and SHA-256 hashes are documented in SOURCES.md and CORPUS_MANIFEST.json.

3.2 Questions and evaluation design

I wrote five questions before generating retrieval results and committed questions.yaml first. This prevents changing the expected answers after seeing which chunks were retrieved. The questions cover SSRF, path traversal, remote code execution, denial of service, affected versions, and remediation.

For each question, I retrieved the top five results from each technique. Recall@5 was counted as successful when the expected source document appeared in the five retrieved source files.

3.3 Chunking techniques

Technique	Configuration	Reason for using it
Token	Chunk size 256 tokens, overlap 40 tokens	Creates focused pieces with controlled overlap
Semantic	Buffer size 1, breakpoint percentile 95	Attempts to place boundaries where the topic changes
Sentence-window	One sentence with a three-sentence surrounding window	Preserves local context around a sentence-level match

All three pipelines used the same public sentence-transformers/all-MiniLM-L6-v2 model. The model produced 384-dimensional vectors. Each technique used an in-memory SimpleVectorStore, so the experiment did not require an external database or hosted vector store.

```

embed_model = HuggingFaceEmbedding(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

index = VectorStoreIndex(
    nodes,
    storage_context=storage_context,
    embed_model=embed_model,
)

retriever = index.as_retriever(
    similarity_top_k=5
)

```

3.4 Retrieval output

For every retrieved result, the program recorded the rank, LlamaIndex store score, explicitly recomputed cosine similarity, source filename, chunk length, preview text, vector shapes, and retrieval latency. The query embedding dimension and first eight values were also saved.

```

PS C:\Projects\data260-8844> Get-Content .\reports\hw03\raw\metrics.json
[
  {
    "technique": "Token",
    "chunks": 2260,
    "avg_chunk_length": 362.36,
    "top1_cosine": 0.655152,
    "mean_at_k_cosine": 0.527624,
    "recall_at_k_mean": 1,
    "mean_retrieval_latency_ms": 153.062
  },
  {
    "technique": "Semantic",
    "chunks": 142,
    "avg_chunk_length": 4856.94,
    "top1_cosine": 0.629102,
    "mean_at_k_cosine": 0.414521,
    "recall_at_k_mean": 1,
    "mean_retrieval_latency_ms": 31.038
  },
  {
    "technique": "Sentence-window",
    "chunks": 727,
    "avg_chunk_length": 947.94,
    "top1_cosine": 0.651468,
    "mean_at_k_cosine": 0.495669,
    "recall_at_k_mean": 1,
    "mean_retrieval_latency_ms": 61.474
  }
]
PS C:\Projects\data260-8844>

```

Official retrieval metrics output.

4. Retrieval results and discussion

Technique	Chunks	Average chunk length	Top-1 cosine	Mean@5 cosine	Recall@5	Latency ms
Token	2,260	362.36	0.655152	0.527624	1.000000	153.062
Semantic	142	4,856.94	0.629102	0.414521	1.000000	31.038
Sentence-window	727	947.94	0.651468	0.495669	1.000000	61.474

All three approaches retrieved the expected source document for every question within the top five results. This is why each technique has a Recall@5 mean of 1.0. That result is encouraging, but it does not mean the first result was always the best answer. The top result can still be a related advisory rather than the exact document required by the question.

Token chunking performed best overall on the similarity measurements. Its smaller chunks separated facts such as affected versions, fixed versions, and vulnerability mechanisms. The 40-token overlap also reduced the chance that an important statement would be split across two unrelated chunks.

Sentence-window chunking was close to token chunking. Its main advantage is context: a sentence-level match can be interpreted with nearby sentences. Semantic chunking was the fastest to search because it produced only 142 chunks, but those chunks were very large. A single embedding represented many different facts, which reduced the average similarity across the top five results.

4.1 High-scoring incorrect retrieval

The strongest incorrect result occurred for question q4, which asked why Open WebUI OAuth profile-picture processing could be exploited for SSRF. The expected document was GHSA-226f-f24g-524w.json, but Semantic chunking ranked GHSA-24c9-2m8q-qhmq.json first with a cosine similarity of 0.7008324.

The incorrect document was not random. It was a closely related Open WebUI SSRF advisory and shared terms such as Open WebUI, OAuth, profile pictures, and SSRF. The embedding therefore judged it highly similar. However, the question asked about the specific redirect-bypass behavior: the initial URL is validated, but the redirect destination is not revalidated. The related advisory did not contain that exact answer.

A similar incorrect top-1 result occurred with Token chunking at cosine similarity 0.67904045. This example shows why a high embedding score should not automatically be treated as proof that the retrieved chunk contains the requested answer.

5. Verification and reproducibility

I created a self-check script that verifies the application files, screenshots, corpus size, question count, manifest, raw retrieval outputs, three techniques, and incorrect-retrieval evidence. The verification passed all checks.

```
PS C:\Projects\data260-8844> .\venv\Scripts\python.exe `
>> .\code\hw3_rag\verify_hw03.py
[PASS] file:SOURCES.md: reports\hw03\SOURCES.md exists and is non-empty
[PASS] file:CORPUS_MANIFEST.json: reports\hw03\CORPUS_MANIFEST.json exists and is non-empty
[PASS] file:questions.yaml: reports\hw03\questions.yaml exists and is non-empty
[PASS] file:METRICS.md: reports\hw03\METRICS.md exists and is non-empty
[PASS] file:RUN_LOG.txt: reports\hw03\RUN_LOG.txt exists and is non-empty
[PASS] file:secure_cookie_header.png: reports\hw03\screenshots\secure_cookie_header.png exists and is non-empty
[PASS] file:idle_timeout_redirect.png: reports\hw03\screenshots\idle_timeout_redirect.png exists and is non-empty
[PASS] file:retrieval_results.jsonl: reports\hw03\raw\retrieval_results.jsonl exists and is non-empty
[PASS] file:retrieval_results.csv: reports\hw03\raw\retrieval_results.csv exists and is non-empty
[PASS] file:query_embeddings.jsonl: reports\hw03\raw\query_embeddings.jsonl exists and is non-empty
[PASS] file:chunking_stats.json: reports\hw03\raw\chunking_stats.json exists and is non-empty
[PASS] file:metrics.json: reports\hw03\raw\metrics.json exists and is non-empty
[PASS] file:wrong_retrieval_candidates.json: reports\hw03\raw\wrong_retrieval_candidates.json exists and is non-empty
[PASS] file:main.py: code\web_application\main.py exists and is non-empty
[PASS] file:auth.py: code\web_application\routers\auth.py exists and is non-empty
[PASS] file:index.html: code\web_application\templates\index.html exists and is non-empty
[PASS] file:login.html: code\web_application\templates\login.html exists and is non-empty
[PASS] file:dashboard.html: code\web_application\templates\dashboard.html exists and is non-empty
[PASS] questions_count: Found 5 questions.
[PASS] corpus_size: Corpus contains 507649 bytes.
[PASS] corpus_document_count: Manifest contains 69 documents.
[PASS] retrieval_row_count: Found 75 retrieval rows; expected 75.
[PASS] three_techniques: Found techniques: ['Semantic', 'Sentence-window', 'Token']
[PASS] incorrect_retrieval_evidence: Found 2 incorrect top-1 candidates.
[PASS] selected_corpus_files: Found 69 selected JSON documents.

Overall verification: True
Output: C:\Projects\data260-8844\reports\hw03\verification.json
PS C:\Projects\data260-8844> |
```

Self-check output showing Overall verification: True.

The repository includes exact instructions in reports/hw03/README.md. The experiment can be rerun with the project virtual environment, the local corpus, the fixed questions, and the saved scripts. The results are stored as JSONL, CSV, JSON metrics, and a timestamped run log.

6. AI use disclosure

I used an AI assistant to understand the homework requirements, plan the repository structure, explain authentication and retrieval concepts, draft starter code, and troubleshoot errors. I personally created the branch, downloaded and selected the corpus, wrote the questions, tested the FastAPI application, captured screenshots, ran the retrieval experiment, and reviewed the results.

One unsuitable suggestion was the first import of MetadataMode directly from llama_index.core. My installed LlamaIndex version required the import from llama_index.core.schema. I detected this from the ImportError

produced by the smoke test. I also independently verified the Hugging Face cache problem when the model download failed with an access-denied error.

I fixed the import path and redirected the Hugging Face cache to a writable `.hf_cache` directory inside the repository. After those changes, the smoke test passed and the full retrieval experiment completed successfully. The longer AI-use explanation is also included in `reports/hw03/AI_USE.md`.

7. Conclusion

This homework connected application security with information retrieval. The authentication portion demonstrated how sessions, protected routes, secure cookies, and timeout checks work in a FastAPI application. The retrieval portion showed that chunking choices affect both the quality and speed of vector search.

For this vulnerability corpus, I consider Token chunking the strongest overall choice because it produced the highest top-1 and Mean@5 cosine scores while still achieving perfect Recall@5. Sentence-window chunking was a close alternative when preserving nearby context is especially important. Semantic chunking was the fastest, but its very large chunks made the similarity results less focused.

Supporting files, raw outputs, source documentation, screenshots, and verification results are committed in the GitHub repository.